verify_qa_evidence

`verify_qa_evidence.sh` — companion guide

Revision: 2 **Last modified:** 2026-05-28T00:00:00Z

Field	Value
Script	`scripts/verify_qa_evidence.sh`
Authority	constitution submodule `Constitution.md` §11.4.83 (docs/qa/ end-user evidence mandate); §11.4.18 (script documentation mandate)
Status	active — ADVISORY by default, ENFORCING (blocking) under `--enforce --since <baseline>` (operator-authorised release gate, HXC-019)
Last verified	2026-05-28

Table of contents

Overview

`scripts/verify_qa_evidence.sh` is the enforcement seam for the §11.4.83 `docs/qa/` end-user evidence mandate. It scans feature-shipping commits and reports when a feature commit has no matching `docs/qa/<run-id>/` directory carrying its end-user evidence transcript.

Two modes:

- **Advisory (default)** — ALWAYS exits 0; prints a warn-mode notice. For ad-hoc visibility; not wired into any git hook.
- **Enforcing** (`--enforce --since <ref>`) — exits **1** on any in-scope violation, **0** when clean. This is the §11.4.83 operative-rule-(5) release gate. The operator **authorised** the blocking gate on 2026-05-28 (HXC-019).

Prerequisites

- Run from inside the HelixCode git repository (any cwd — the script resolves the repo root via `git rev-parse --show-toplevel`).
- `git` and POSIX coreutils on `PATH`.
- `bash` (the shebang is honest — the script uses bash and `set -u`).

Usage examples

Scan the default window (last 20 commits):

```
scripts/verify_qa_evidence.sh
```

Scan a custom window (last 50 commits):

```
scripts/verify_qa_evidence.sh 50
```

Expected output is a per-commit advisory report ending with a `RESULT: advisory scan complete (exit 0 — warn-mode).` line.

Run the enforcing release gate scoped to the convention baseline:

```
scripts/verify_qa_evidence.sh --enforce --since ed84f90e
```

Exits 0 when every in-range feature commit carries its `docs/qa/<run-id>/` directory; exits 1 (with per-commit `VIOL` lines on stderr) on any violation.

Enforcing mode + baseline scoping

`--enforce` evaluates only the range `<since>..HEAD` — commits that are descendants of the `--since` baseline by merge-ancestry (not author-date sorting). The baseline is the commit that introduced the convention by **adding** `docs/qa/README.md`:

```
git log --diff-filter=A --format='%H %cI %s' -- docs/qa/README.md
# ed84f90e  2026-05-28T16:09:55+03:00  feat(qa): establish docs/qa/ ... (H
```

`--since` is **mandatory** in enforcing mode. Running `--enforce` without it is a misuse error (exit 2): enforcing over the whole history would block on thousands of pre-convention legacy feature commits and make `HEAD` un-releasable. `--since` accepts any git revision (SHA, tag) **or** an approxidate such as `2026-05-28`.

Per-commit opt-out

A commit whose message (subject **or** body) contains the literal token `[no-qa-evidence]` is **exempt** from the gate. Use it for a change that trips the feature-shipping heuristic but ships no user-facing feature (pure refactor, governance-only, doc-only). It is the single documented escape hatch; an untagged feature commit with no evidence directory always fails the enforcing gate.

Edge cases

- **Not in a git repo** — prints a notice and exits 0.

- **No `docs/qa/` tree** — every feature commit is reported as a WARN (no run-id directories exist to match against); still exits 0.
- **Pure refactor that touches watched paths** — may produce a false positive WARN. Acceptable for a warn-mode advisory notice.
- **Run-id matched by commit subject** — a commit whose subject contains a known `docs/qa/<run-id>/` directory name (e.g. `HXC-019`) is reported `ok`.

Internal behaviour

1. Resolves repo root and `cd`s into it.
2. Enumerates existing `docs/qa/<run-id>/` directories.
3. Determines the commit window — advisory: the last N commits; enforcing: the `<since>..HEAD` range.
4. A commit is "feature-shipping" if it touches non-test code under `helix_code/internal/**`, `helix_code/cmd/**`, or `helix_code/applications/**` (excluding `*_test.go`). File listing uses `git diff-tree --no-commit-id --name-only -r` (robust across git versions).
5. A commit carrying `[no-qa-evidence]` is reported `exempt` and skipped.
6. For each remaining feature commit, checks whether its subject references a known run-id directory; prints `ok` / `WARN` (advisory) or `ok` / `VIOL` (enforcing).
7. Advisory: prints a summary and ALWAYS exits 0. Enforcing: exits 1 if any `VIOL` was recorded, else 0.

Release-gate wiring + meta-test

Per the §11.4.83 mandate wording ("release gates"), the enforcing gate is wired into the release gate **only** — never into pre-commit / pre-push hooks:

- `scripts/gates/qa_evidence_gate.sh` — release-gate wrapper that runs `verify_qa_evidence.sh --enforce --since <baseline>` and propagates the exit code. Baseline overridable via `$QA_EVIDENCE_BASELINE` or `$1`.
- `scripts/release-gate-test.sh` — invokes the wrapper; a non-zero result forces the whole release gate red regardless of the Go-test outcome.
- `scripts/tests/verify_qa_evidence_meta_test.sh` — §1.1 paired-mutation meta-test in a throwaway temp git repo, asserting the gate FAILs when evidence is missing, PASSes when present, and honours the `[no-qa-evidence]` opt-out and the mandatory `--since` baseline.

Related scripts

- `docs/qa/README.md` — the convention this script enforces.
- `scripts/gates/qa_evidence_gate.sh` — release-gate wrapper.
- `scripts/tests/verify_qa_evidence_meta_test.sh` — §1.1 meta-test.
- `scripts/release-gate-test.sh` — meta-repo release gate that calls it.
- `scripts/verify-governance-cascade.sh` — the cascade gate exercised in the `docs/qa/HXC-016/` seed example.